

Algorithm Project Report

Topic : Smart City Transportation System .

By :

Ahmed Mohamed Fawzy , ID:23101889

Abanob Yousry Fahmy , ID:23101861

Mahmoud Ismail Elsaid Ismail , ID:23101596

Belal Ashraf Mostafa zaki , ID:23101559

Ahmed Tharwet abd elhamed , ID:23101351

1. Project Overview

A Streamlit-based web application that models the Cairo metropolitan transportation network and applies multiple algorithmic paradigms — MST, shortest path, dynamic programming, greedy, and machine learning — to solve real-world urban transportation problems.

Layer	Technology	Purpose
Frontend	Streamlit ≥ 1.28	Interactive multi-page dashboard
Visualization	Plotly ≥ 5.18	Geographic Mapbox maps + charts
Graph Engine	NetworkX ≥ 3.2	Graph utilities
ML	scikit-learn ≥ 1.3	Traffic prediction models
Data	NumPy ≥ 1.24 , Pandas ≥ 2.0	Numerical computation
Deployment	Docker + docker-compose	Containerized deployment

2. Architecture & Project Structure

Directory	Description
Application	Main home page of app with metrics, network preview and
pages/	Feature pages (Infrastructure, Traffic, Emergency, Transit, Signals, ML, Comparison)
src/algorithms/	Core algorithm implementations Including Greedy , Dynamic Programming , Shortest Path and MST
src/models/	Graph data structure with BPR congestion model
src/data/	All Cairo transportation datasets
src/visualization/	Plotly Mapbox maps + performance charts
tests/	Unit tests for all algorithm Files
Config	Dockerfile, docker-compose, requirements, Streamlit config

3.Algorithms — Detailed Analysis

1.Minimum Spanning Tree — mst.py

Data Structure: UnionFind

- Path compression + union by rank
- Amortized $O(\alpha(n))$ per operation (inverse Ackermann)
- Tracks connected component count

kruskal mst() — $O(E \log E)$ time, $O(V+E)$ space

1. Compute modified weights for all edges
2. Sort edges by weight (ascending)
3. Greedily add edges via Union-Find (skip if cycle)
4. Stop at $V-1$ edges

prim mst() — $O(E \log V)$ time, $O(V+E)$ space

1. Start from node "3" (Downtown Cairo)
2. Push all edges from current node to min-heap
3. Pop minimum, add if destination unvisited
4. Repeat until all nodes visited

Modified Weight Function: compute_mst_weight()

$$\text{Weight} = \frac{\text{effective_distance} \times \text{condition_penalty}}{\text{population_factor} \times \text{facility_factor}}$$

Factor	Formula	Effect
Condition penalty	$1 + (10 - \text{condition}) \times 0.1$	Worse roads → higher weight
Population factor	$1 + \text{pw} \times (\text{avg_pop} / 550000)$	Dense areas → lower weight (prioritized)

Factor	Formula	Effect
Facility factor	1 + bonus if critical type	Hospitals/gov → lower weight
Cost blending (potential)	$\text{dist} \times (1-\alpha) + (\text{cost}/20) \times \alpha$	$\alpha=0.3$ balances distance vs cost

Returns: mst_edges, total_distance, total_cost, connected, execution_time, steps

2 Shortest Path — shortest_path.py

dijkstra() — $O((V+E) \log V)$ time, $O(V)$ space

- Standard binary-heap implementation
- Supports 3 weight modes: distance, time (BPR), congestion
- Early termination when target is reached

astar_search() — $O((V+E) \log V)$ worst, typically better

- Heuristic: **Haversine distance** (admissible since straight-line $\leq$ road distance)
- Emergency mode: **0.6× preemption factor** on all edge costs
- When time_period is set, heuristic is scaled: $h = \text{haversine} / 40 \times 60$ (assumes 40 km/h)

dijkstra_with_closures() — Road closure rerouting

- Accepts closed_edges list of (from, to) tuples
- Builds $O(1)$ lookup set (both directions) to skip closed roads
- Falls back to time-dependent BPR weights

compare_algorithms() — Convenience wrapper

Returns side-by-side results for Dijkstra, A*, and A*-Emergency.

3 Dynamic Programming — dynamic_programming.py

optimize_public_transit_scheduling() — $O(L \times C \times M)$

Problem: Bounded knapsack — allocate C capacity units across L transit lines to maximize total passengers served.

State: $dp[i][j]$ = max passengers using first i lines with j capacity units

Transition: $dp[i][j] = \max_{k \in [0, \min(j, M)]} (dp[i-1][j-k] + \text{passengers}(\text{line}_i, k))$

Passenger model: Diminishing returns — $\text{max_pass} \times (1 - e^{(-0.15 \times \text{effective_units})})$

Detail	Value
Metro multiplier	1 train = 10 bus capacity units
Metro efficiency	1.2× passengers per unit vs bus
Demand source (bus)	OD-matrix sum × 1.5, fallback to daily_passengers
Demand source (metro)	daily_passengers × 1.2
Backtracking	Full allocation recovery per line

road_maintenance_allocation() — $O(R \times (B/G)^2)$

Problem: Another bounded knapsack — distribute budget B across R roads in granularity G increments.

Improvement formula: $\text{max_improvement} \times (1 - e^{(-0.5 \times \text{investment_units})})$

Where $\text{max_improvement} = \text{condition_gap} \times \text{distance} \times (\text{capacity} / 1000)$

MemoizedRoutePlanner — $O(1)$ cached / $O((V+E)\log V)$ new

- Dictionary cache keyed by (source, target, time_period)
- Bounded at 1,000 entries (full eviction on overflow)
- Tracks hits, misses, evictions, and hit rate

analyze_transfer_points() — $O(B \times S + M \times S)$

Identifies multi-modal hubs where Metro and Bus lines intersect. Returns hubs sorted by $\text{hub_complexity_score} = \text{num_metro} + \text{num_bus}$.

4 .Greedy Algorithms — greedy.py

optimize traffic signals() — $O(V \times E)$

Strategy: At each intersection, allocate green time proportional to incoming traffic flow.

For each node with ≥ 2 incoming roads:

1. Collect incoming flow data for current time period
2. Compute proportional green time (min 10s per approach)
3. Subtract 3s yellow per phase
4. Normalize if total exceeds cycle length

Reverse-flow estimation: When no explicit reverse data exists, estimates 70% of forward flow (models Cairo's asymmetric commute patterns).

Optimality: Optimal for isolated intersections, suboptimal for coordinated signal corridors.

emergency vehicle preemption() — $O(P \times E)$

For each intersection on the emergency path:

1. Compute intersection degree and average congestion
2. Estimate normal wait: $\text{cycle} \times \text{phase_fraction} \times \text{congestion_multiplier}$
3. Preempted wait: fixed 5 seconds
4. Delay saved = normal - preempted

5 ML Prediction — ml_prediction.py

Class: TrafficPredictor

Features (5): time_period_encoded, distance, capacity, condition, distance×capacity (interaction)

Target: traffic flow (vehicles/hour)

Models: RandomForestRegressor(n=100, depth=8) / GradientBoostingRegressor(n=100, depth=5)

Training pipeline :

1. 80/20 train/test split
2. StandardScaler fit on train only
3. 5-fold cross-validation on training set
4. Evaluate on held-out test set
5. Refit on full dataset for production

Metrics returned: CV R^2 , test R^2 , MAE, RMSE, feature importances, sample counts.

4-UI Pages

#	Page	Key Features
1	Infrastructure Design	Kruskal/Prim side-by-side, parameter sliders, MST edge detail, weight formula explanation
2	Traffic Optimization	Standard vs time-dependent Dijkstra, road closure rerouting, congestion map, traffic flow charts
3	Emergency Response	A* emergency vs normal routing, A* vs Dijkstra comparison, signal preemption analysis
4	Public Transit	Bus+Metro DP scheduling, transfer point analysis, road maintenance DP, memoized routing demo
5	Traffic Signals	Greedy signal optimization, per-intersection pie charts, optimality analysis
6	ML Prediction	Train RF/GB models, predict all roads, feature importance chart, custom prediction form
7	Algorithm Comparison	MST param sweep, shortest path across periods, full algorithm summary table

5-Test Modules

Module	Tests	What's Verified
test_mst.py	4	Union-Find correctness, Kruskal/Prim connectivity ($V-1$ edges), cross-algorithm consistency ($<50\text{km}$ diff)
test_shortest_path.py	7	Basic Dijkstra path, all 4 time periods, A* pathfinding, A* explores $\leq$ Dijkstra nodes, emergency preemption faster, no-path cases
test_dp.py	4	Bus allocation improvement $\geq 0\%$, unified transit scheduling (Bus+Metro types), maintenance budget $\leq$ total, memoized cache hit rate
test_greedy.py	3	Signals produce positive green time, all periods work, emergency preemption saves delay

6-Complexity Summary

Algorithm	Time	Space	Optimality
Kruskal's MST	$O(E \log E)$	$O(V + E)$	Optimal (modified weights)
Prim's MST	$O(E \log V)$	$O(V + E)$	Optimal (modified weights)
Dijkstra's	$O((V+E) \log V)$	$O(V)$	Optimal (non-negative weights)
A* Search	$O((V+E) \log V)$	$O(V)$	Optimal (admissible heuristic)
Transit Scheduling DP	$O(L \times C \times M)$	$O(L \times C)$	Optimal
Road Maintenance DP	$O(R \times (B/G)^2)$	$O(R \times B/G)$	Optimal
Signal Optimization	$O(V \times E)$	$O(V)$	Locally optimal
Emergency Preemption	$O(P \times E)$	$O(P)$	Heuristic
ML Prediction	$O(N \times T \times D)$	$O(N \times D)$	N/A (prediction)

